Experiment Report

Course: Computer Programming

Experiment 3: Functions

Class:

Name:

Student ID:

Instructor:

Date: 2026.6.18

Experiment 3: Functions

I. Objectives and Content

This experiment aims to practice C++ functions, which is chapter 6 of the textbook. The detailed goals of this experiment are as follows.

- Modularity: Understand the benefits of dividing a program into smaller, manageable tasks using functions.
- Function Elements: Master the three essential parts: Function Declaration (Prototype), Definition, and Call.
- Data Exchange: Understand the mechanism of passing parameters and returning values.
- Library vs. User-defined: Utilize C++ standard library functions (math, cmath) and create custom user-defined functions.
- Scope & Lifetime: Distinguish between local and global variables and understand their visibility in memory.

Complete the following 8 programming tasks. Ensure all code includes proper documentation and follows the specific technical requirements for each problem.

II. Preparation

This preparation focuses on reviewing the basic knowledge and programming skills required for Experiment 3. Before completing the programming tasks, the relevant concepts of C++ functions were previewed, including function declaration, function definition, function call, parameter passing, return values, and variable scope. The preparation also included reviewing the use of standard library functions from <cmath> and formatted output with <iomanip>. In addition, the Visual Studio Code environment and C++ compiler were checked to ensure that each program could be compiled and executed correctly. Through this preparation, the logical structure, input and output requirements, and testing methods for each task were clarified.

III. Project Results

Task 1: Greeting Function (Void, No Parameters)

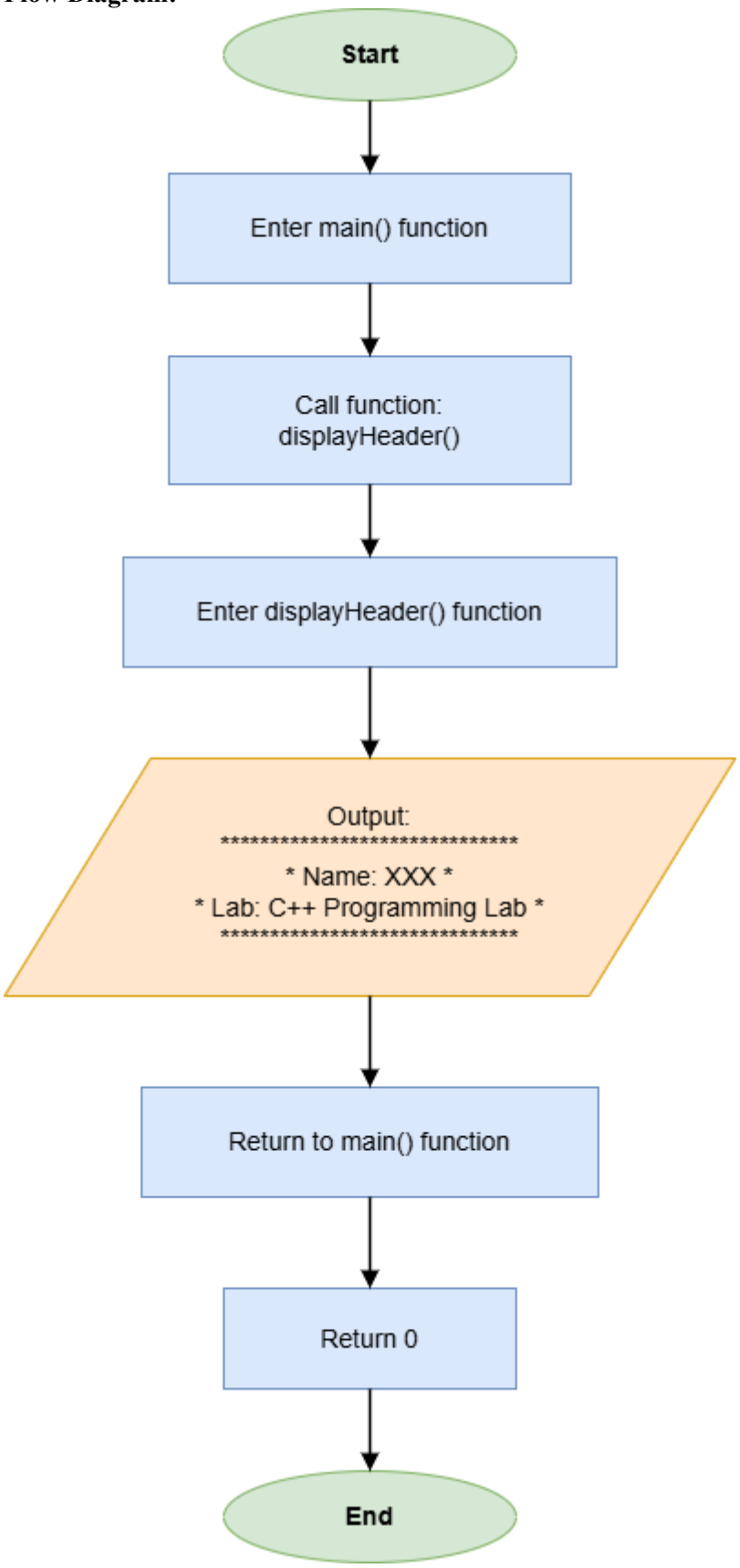
1. **Description**: Describe the program logic in words or with a flow diagram. Do not paste the source code here.
   1) Write the void function displayHeader() that prints my name and lab info.

```
void displayHeader()
{
    cout << "*****************************" << endl;
    cout << "*Name: XXX                  *" << endl;
    cout << "*Lab: C++ Programming Lab    *" << endl;
    cout << "*****************************" << endl;
}
```

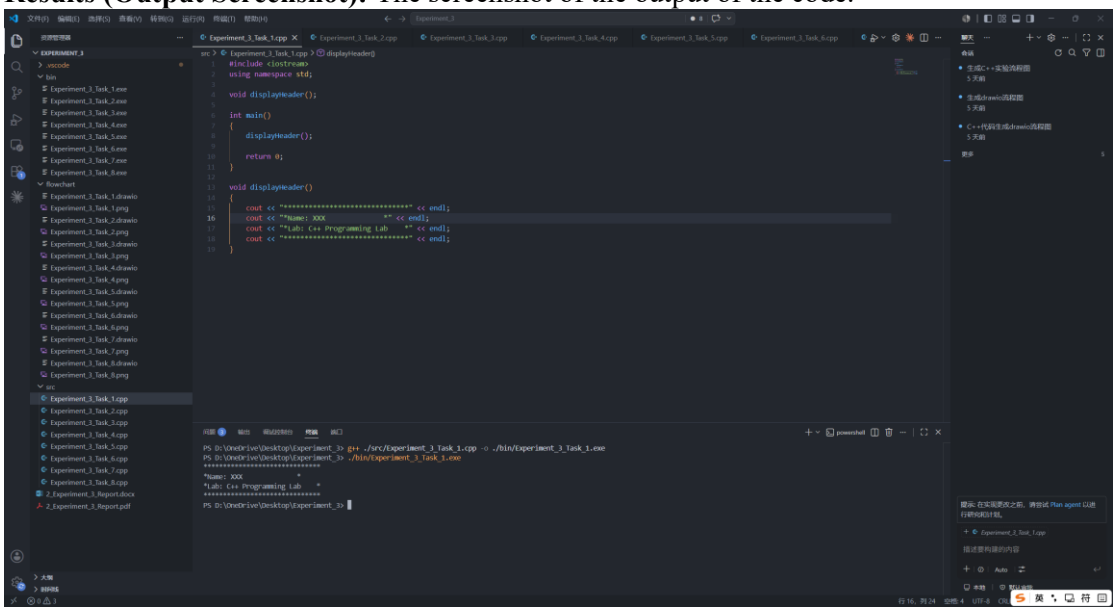
   2) Call it in the main function.

```
int main()
{
    displayHeader();

    return 0;
}
```

2. Flow Diagram:



3. **Results (Output Screenshot):** The screenshot of the output of the code.



Task 2: Square and Cube (Data-Returning, One Parameter)

1. **Description:**

1) Create a function long calculatePower(int base, int exp) that returns $base^{exp}$.

```
long calculatePower(int base, int exp)
{
    long result = 1;
    for (int i = 0; i < exp; i++)
    {
        result *= base;
    }
    return result;
}
```

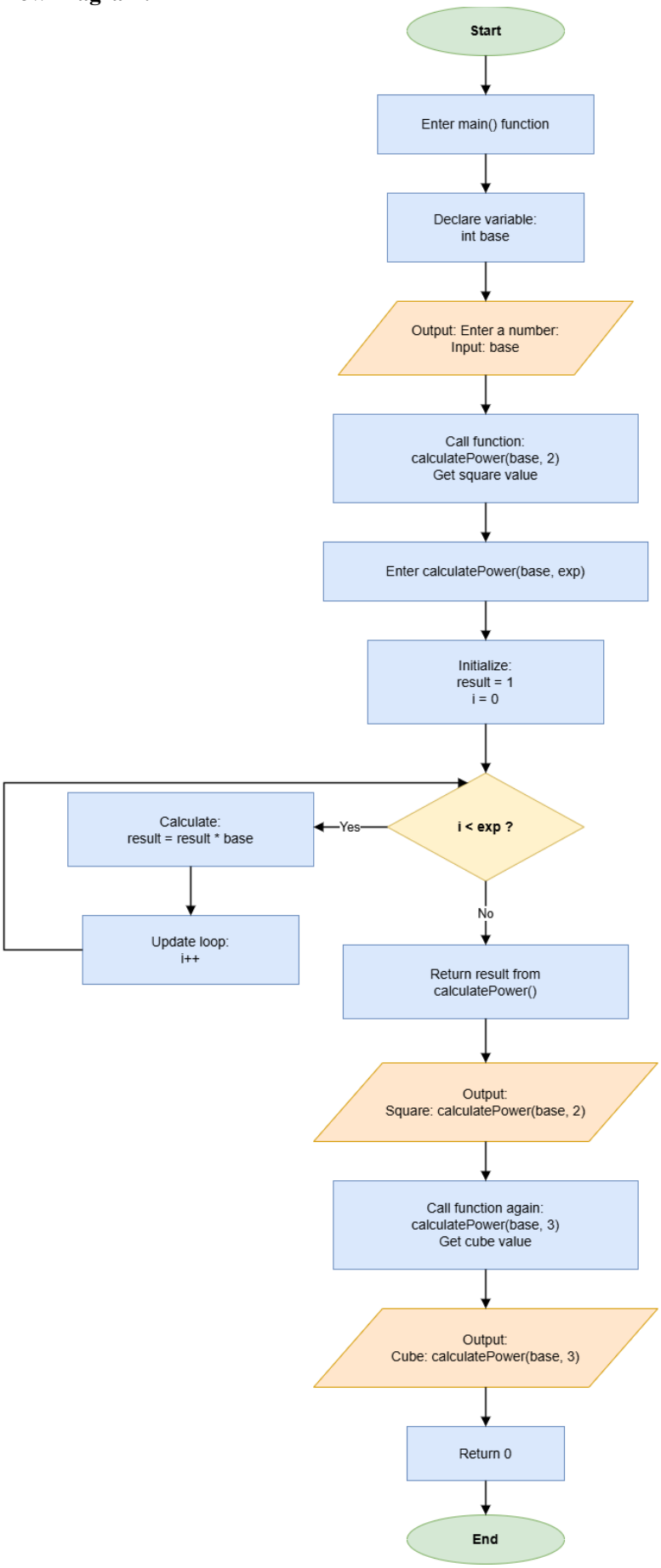
2) Input the number base.

```
    int base;
    cout << "Enter a number: ";
    cin >> base;
```

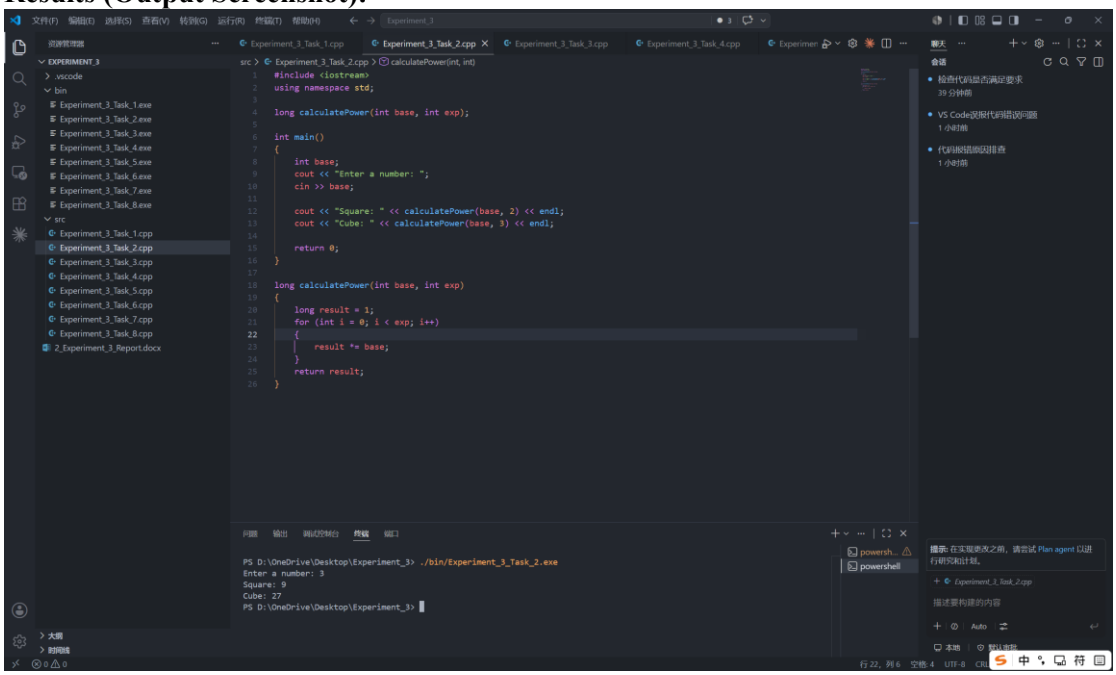
3) Print the square and the cube.

```
    cout << "Square: " << calculatePower(base, 2) << endl;
    cout << "Cube: " << calculatePower(base, 3) << endl;
```

2. Flow Diagram:



3. Results (Output Screenshot):



Task 3: Maximum of Three (Comparison Logic)

1. **Description:**
   1) Write a function int findMax(int n1, int n2, int n3) that returns the largest of the three integers.

```
int findMax(int n1, int n2, int n3)
{
    if (n1 >= n2 && n1 >= n3)
    {
        return n1;
    }
    else if (n2 >= n1 && n2 >= n3)
    {
        return n2;
    }
    else
    {
        return n3;
    }
}
```

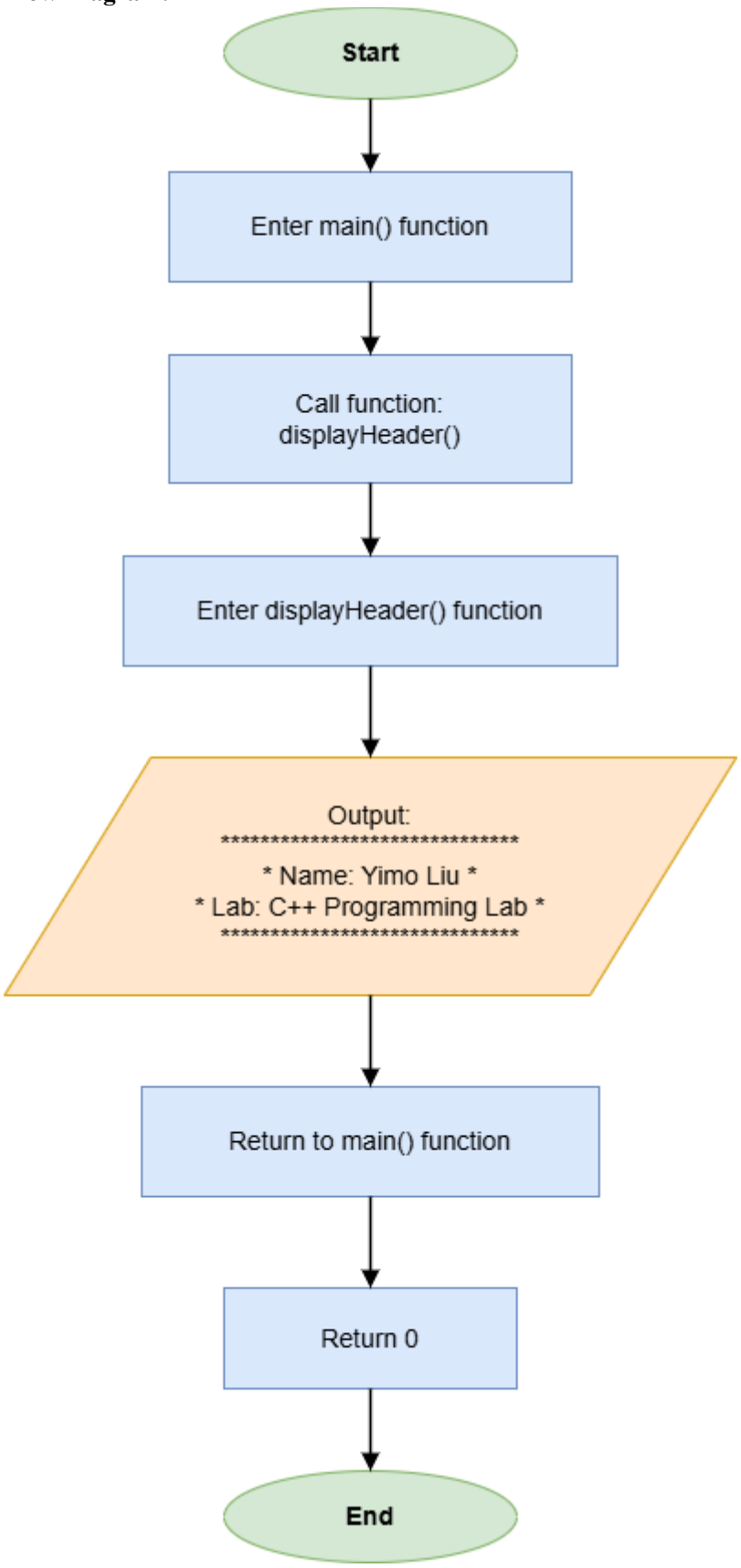
   2) Input the three integers.

```
    int n1, n2, n3;
    cout << "Enter three numbers: ";
    cin >> n1 >> n2 >> n3;
```

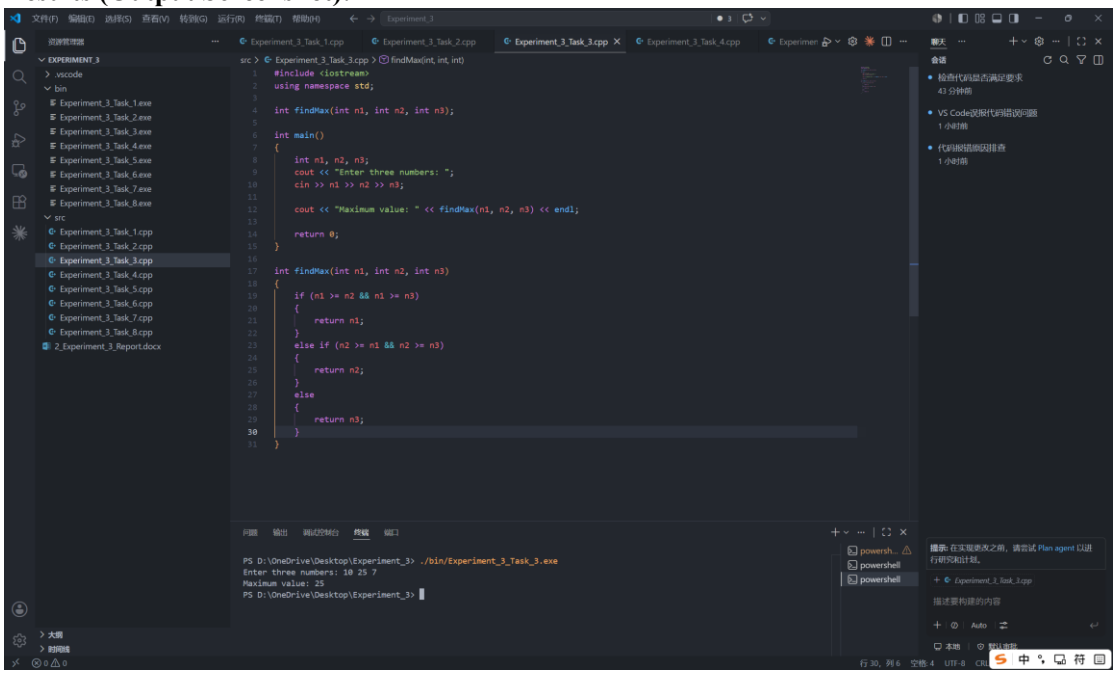
   3) Call findMax() from main and display the returned maximum value.

```
    cout << "Maximum value: " << findMax(n1, n2, n3) << endl;
```

2. **Flow Diagram:**



3. Results (Output Screenshot):



Task 4: Circle Area Calculator (Constants & Math)

1. **Description:**
   1) Write a function double getCircleArea(double radius) that returns the area of a circle using $A = \pi r^2$.

```
double getCircleArea(double radius)
{
    const double PI = 3.14159;
    return PI * radius * radius;
}
```

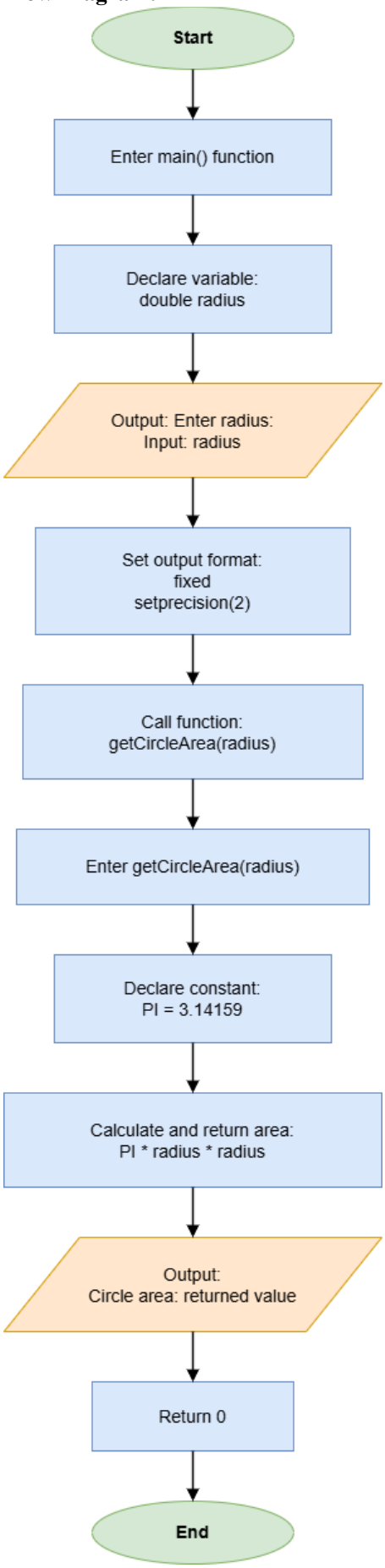
   2) Input the radius.

```
    double radius;
    cout << "Enter radius: ";
    cin >> radius;
```

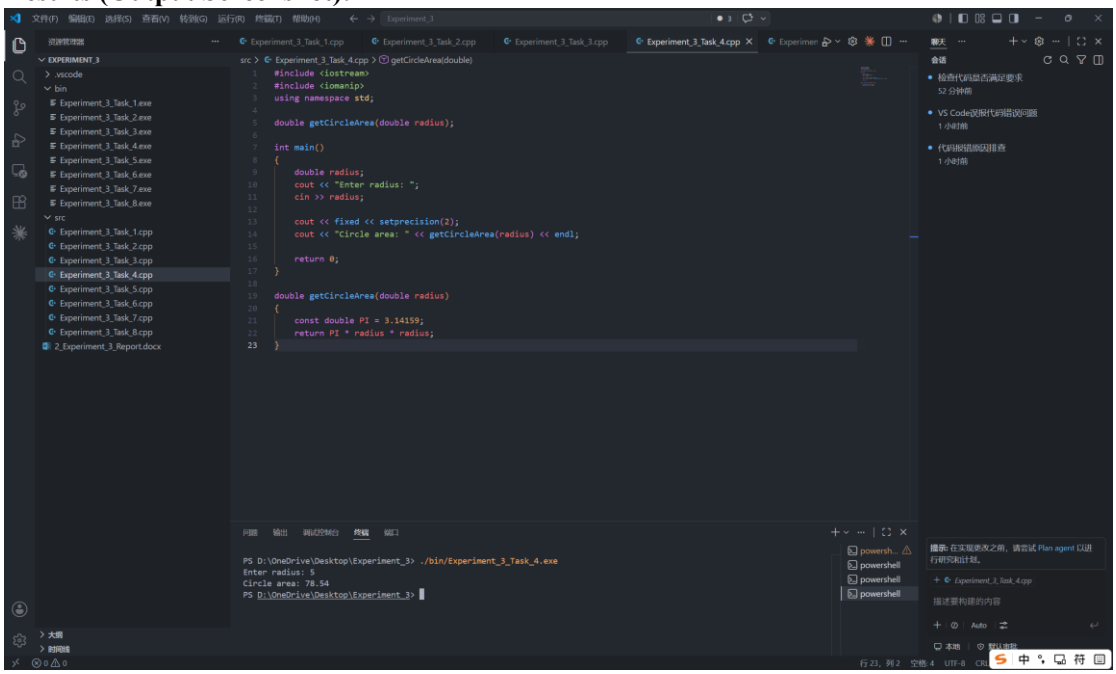
   3) Call getCircleArea() from main() and print the area of the circle with setpricision(2).

```
    cout << fixed << setprecision(2);
    cout << "Circle area: " << getCircleArea(radius) << endl;
```

2. Flow Diagram:



3. Results (Output Screenshot):



Task 5: Temperature Converter (Multiple Functions)

1. **Description:**

1) Create two functions: celsiusToFahrenheit() and fahrenheitToCelsius().

```
double celsiusToFahrenheit(double celsius)
{
    return (celsius * 9.0 / 5.0) + 32;
}

double fahrenheitToCelsius(double fahrenheit)
{
    return (fahrenheit - 32) * 5.0 / 9.0;
}
```

2) Print the menu and allow the user to choose and input the temperature.

```
    cout << "Select conversion:" << endl;
    cout << "1. Celsius to Fahrenheit" << endl;
    cout << "2. Fahrenheit to Celsius" << endl;

    int choice, temperature;
    cout << "Enter choice: ";
    cin >> choice;
    cout << "Enter temperature: ";
    cin >> temperature;
```

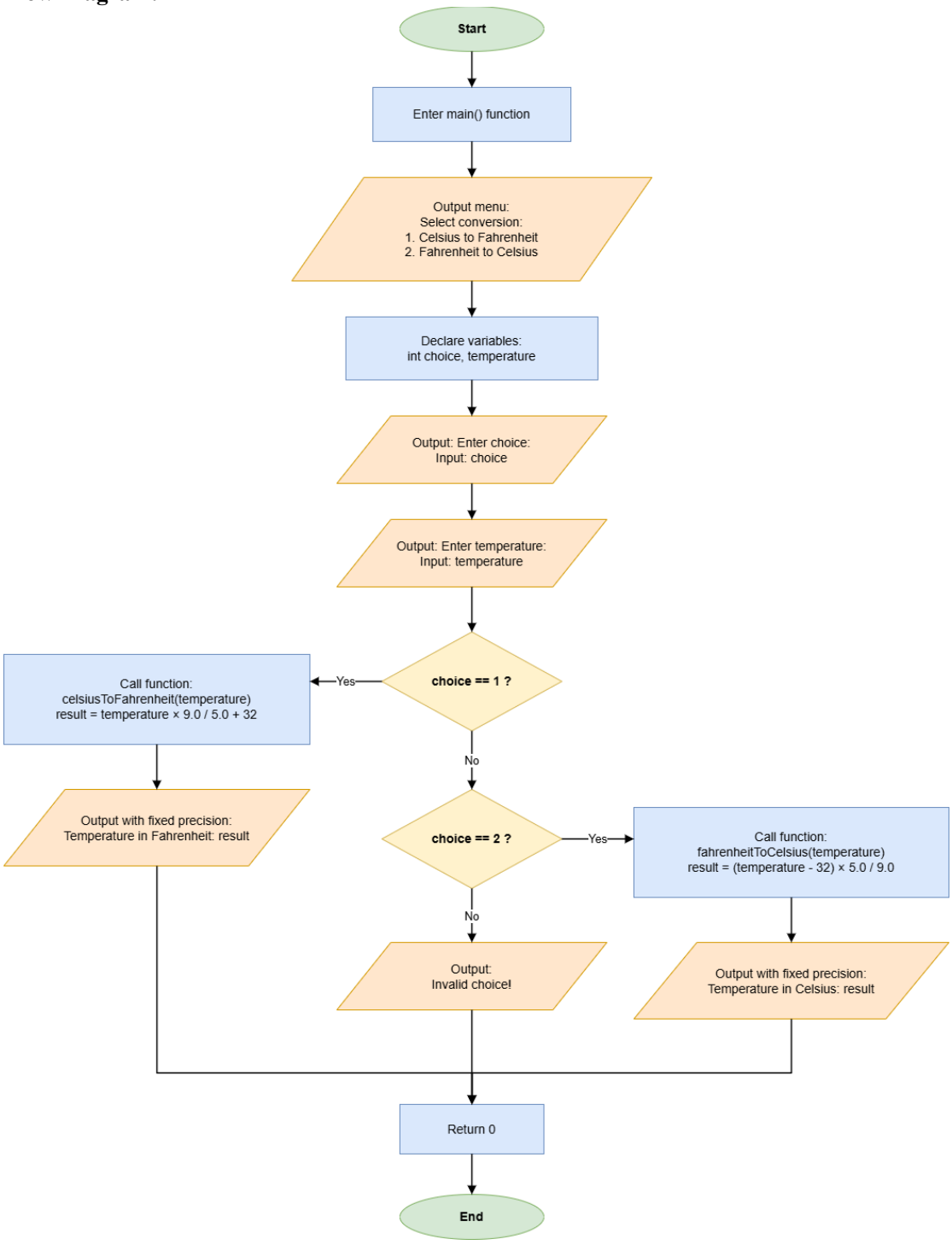
3) Call the two functions from main() depending on the user's choice.

```
    if (choice == 1)
    {
        cout << fixed << setprecision(2);
        cout << "Temperature in Fahrenheit: " <<
celsiusToFahrenheit(temperature) << endl;
    }
    else if (choice == 2)
    {
        cout << fixed << setprecision(2);
        cout << "Temperature in Celsius: " <<
fahrenheitToCelsius(temperature) << endl;
    }
```

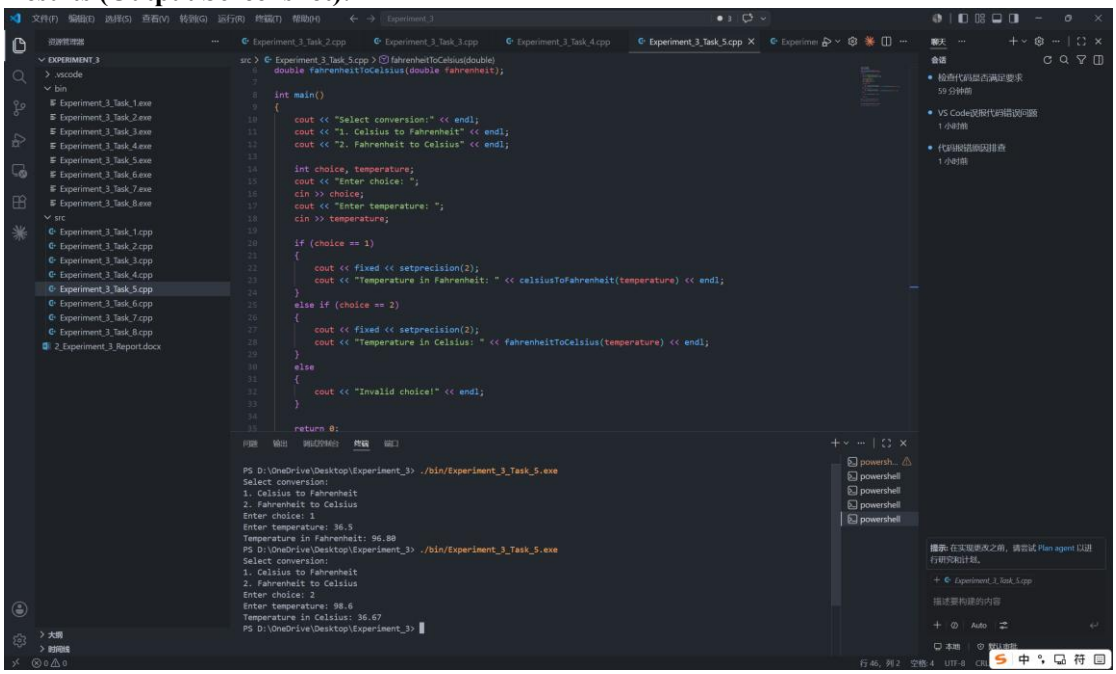
4) Protection from invalid choice.

```
    else
    {
        cout << "Invalid choice!" << endl;
    }
```

2. Flow Diagram:



3. Results (Output Screenshot):



Task 6: Prime Number Checker (Boolean Function)

1. **Description:**
   1) Write a function bool isPrime(int n) that returns true if a number is prime and false otherwise.

```
bool isPrime(int n)
{
    for (int i = 2; i <= n / 2; i++)
    {
        if (n % i == 0)
            return false;
    }
    return true;
}
```

   2) Input the number to be checked.

```
    int number;
    cout << "Enter a number: ";
    cin >> number;
```

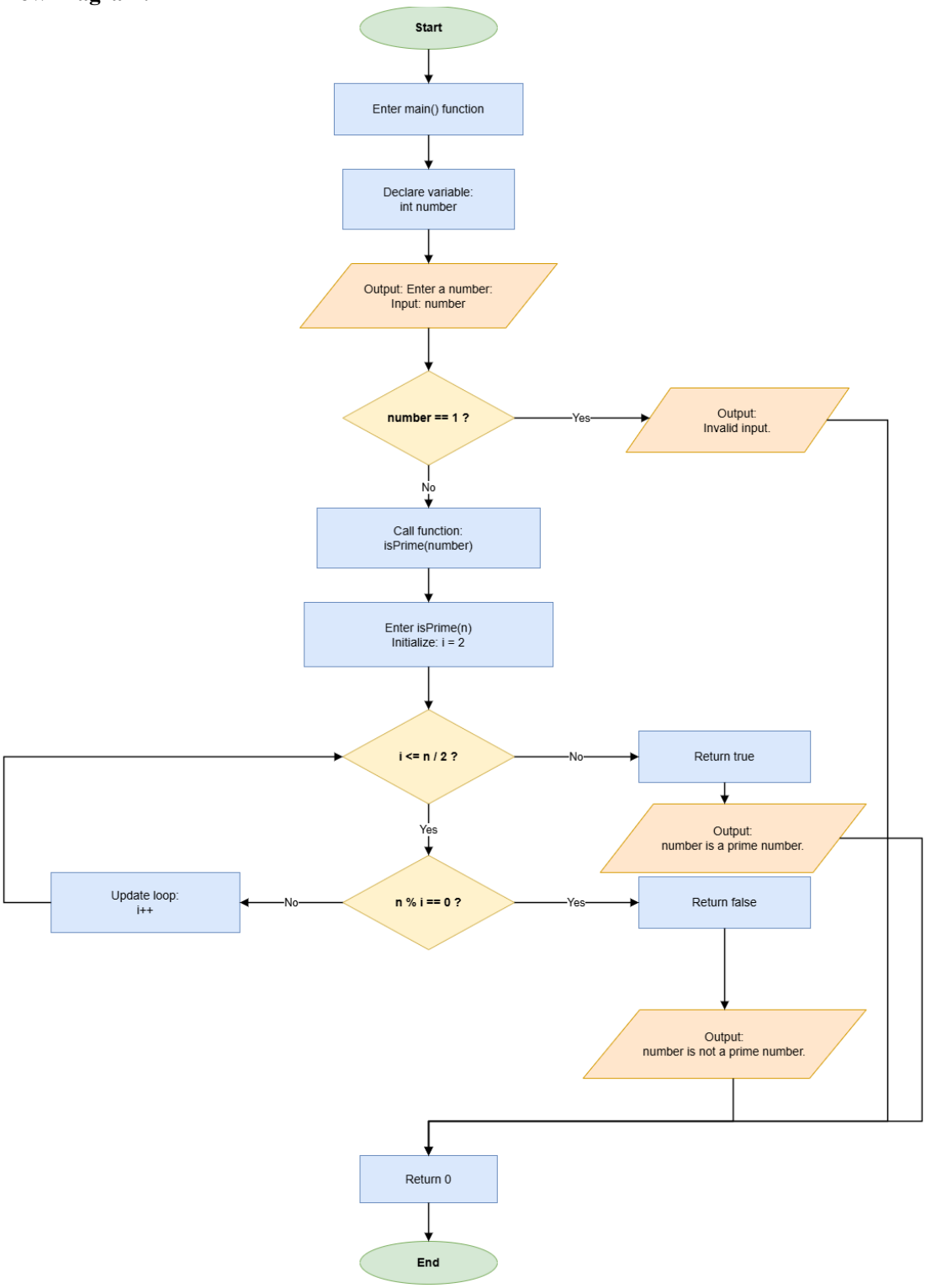
   3) Protection when number is 1.

```
    if (number ==1)
    {
        cout << "Invalid input." << endl;
    }
```

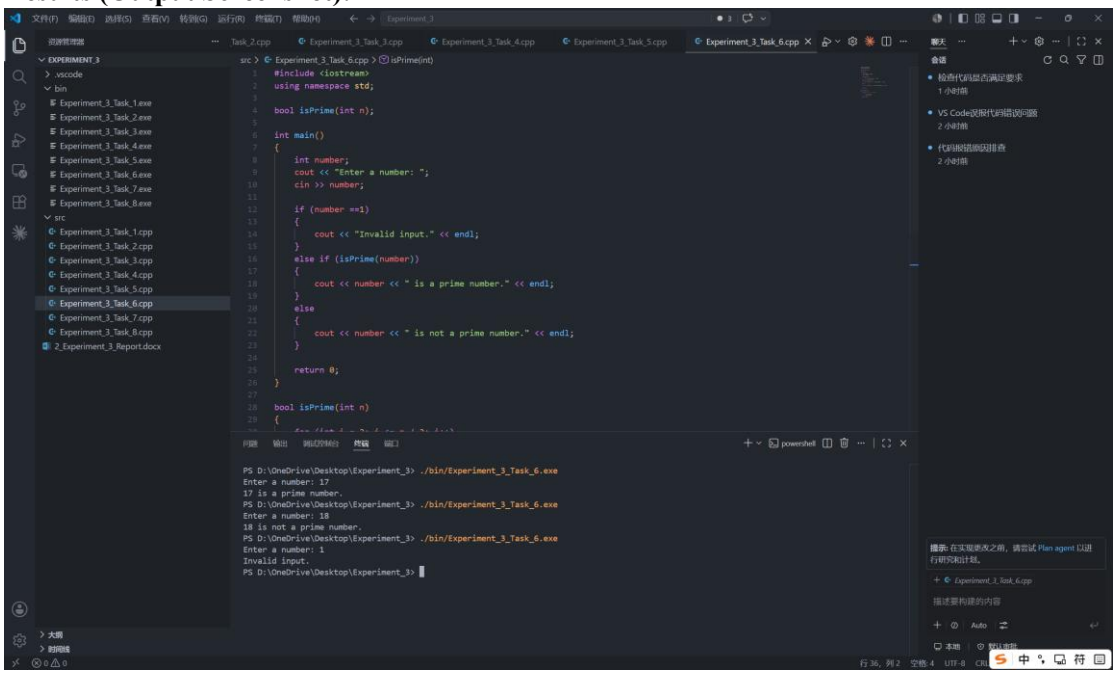
   4) Call isPrime() in main() to check the user's input.

```
    else if (isPrime(number))
    {
        cout << number << " is a prime number." << endl;
    }
    else
    {
        cout << number << " is not a prime number." << endl;
    }
```

2. Flow Diagram:



3. Results (Output Screenshot):



Task 7: Mathematical Library Exploration (cmath)

1. **Description:**

1) Input a positive number.

```
double n;
cout << "Enter a number: ";
cin >> n;
```

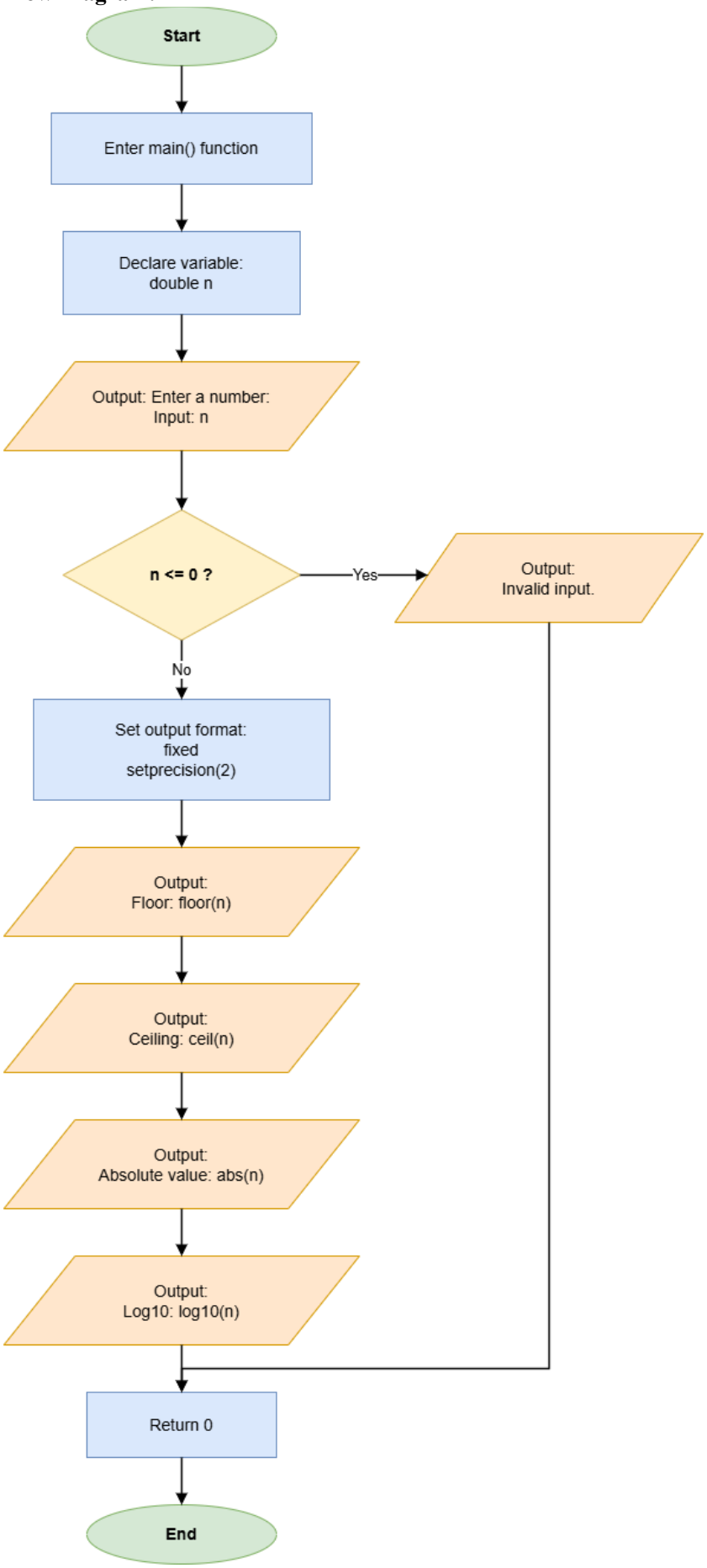
2) When the input is not positive, print “Invalid output.”

```
if (n <= 0)
{
    cout << "Invalid input." << endl;
}
```

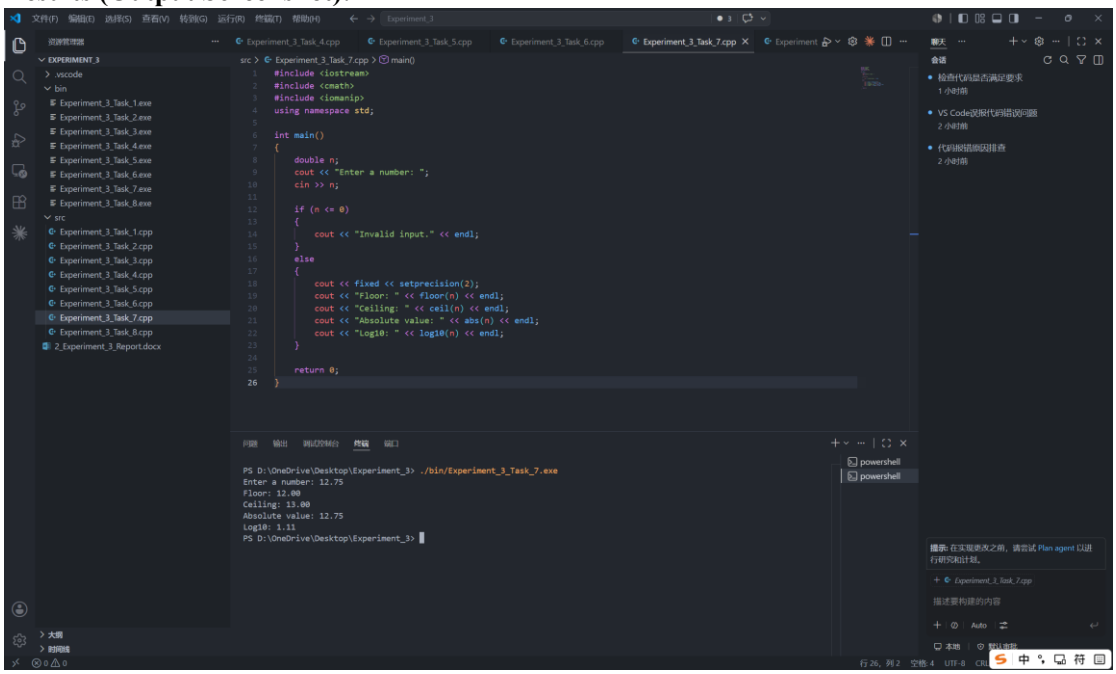
3) Use floor(), ceil(), abs(), and log10() to process the input number.

```
else
{
    cout << fixed << setprecision(2);
    cout << "Floor: " << floor(n) << endl;
    cout << "Ceiling: " << ceil(n) << endl;
    cout << "Absolute value: " << abs(n) << endl;
    cout << "Log10: " << log10(n) << endl;
}
```

2. Flow Diagram:



3. Results (Output Screenshot):



Task 8: Simple Menu System (Modular Program)

1. **Description:**

1) Create the three functions.

```
long calculatePower(int base, int exp)
{
    long result = 1;
    for (int i = 0; i < exp; i++)
    {
        result *= base;
    }
    return result;
}

double getCircleArea(double radius)
{
    const double PI = 3.14159;
    return PI * radius * radius;
}

bool isPrime(int n)
{
    for (int i = 2; i <= n / 2; i++)
    {
        if (n % i == 0)
            return false;
    }
    return true;
}
```

2) Print the menu.

```
    cout << "Menu:" << endl;
    cout << "1. Power Calculation" << endl;
    cout << "2. Circle Area" << endl;
    cout << "3. Prime Check" << endl;
```

3) Input the choice.

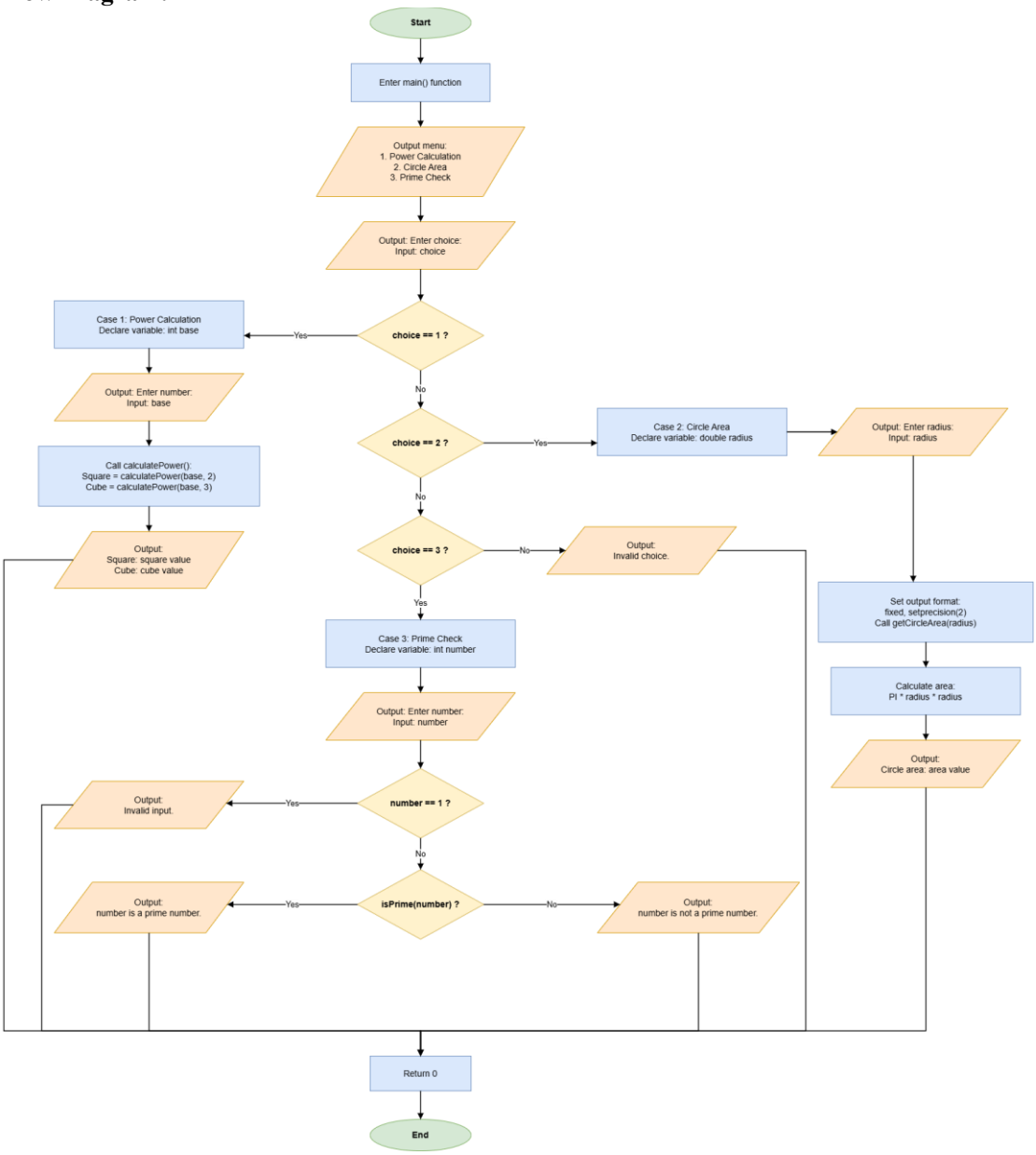
```
    int choice;
    cout << "Enter choice: ";
    cin >> choice;
```

4) Use switch-case to call different functions depending on the user's choice.

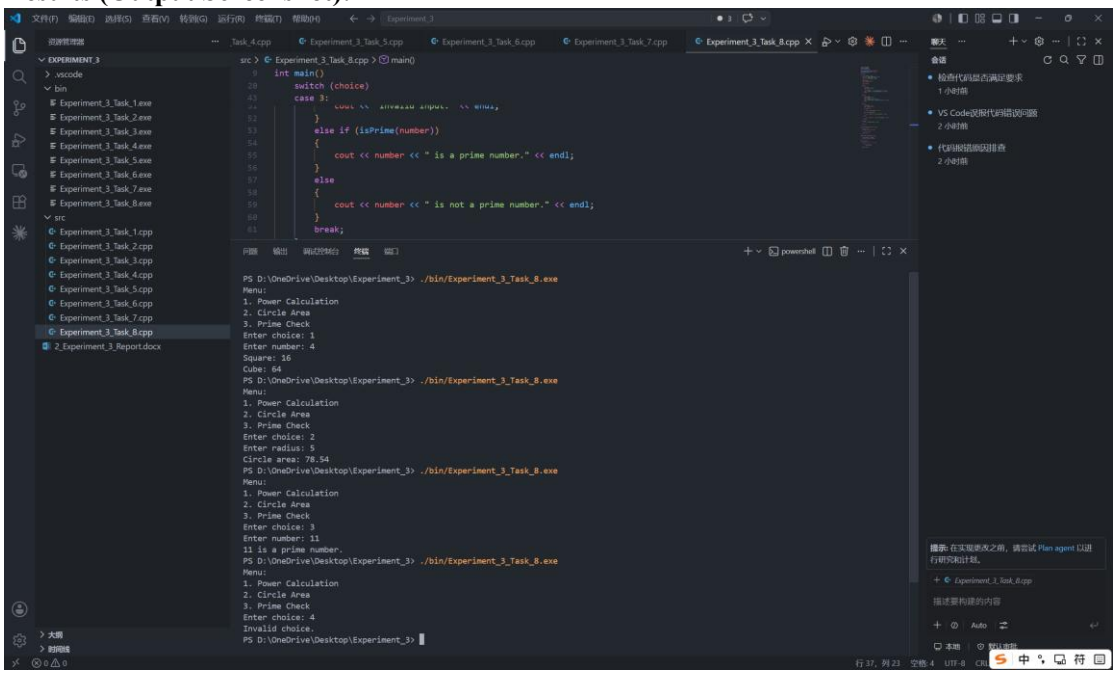
```
    switch (choice)
    {
    case 1:
    {
        int base;
        cout << "Enter number: ";
        cin >> base;

        cout << "Square: " << calculatePower(base, 2) << endl;
        cout << "Cube: " << calculatePower(base, 3) << endl;
        break;
```

```
    }

    case 2:
    {
        double radius;
        cout << "Enter radius: ";
        cin >> radius;
        cout << fixed << setprecision(2);
        cout << "Circle area: " << getCircleArea(radius) << endl;
        break;
    }

    case 3:
    {
        int number;
        cout << "Enter number: ";
        cin >> number;

        if (number == 1)
        {
            cout << "Invalid input." << endl;
        }
        else if (isPrime(number))
        {
            cout << number << " is a prime number." << endl;
        }
        else
        {
            cout << number << " is not a prime number." << endl;
        }
        break;
    }
    default:
        cout << "Invalid choice." << endl;
    }
```

2. Flow Diagram:



3. Results (Output Screenshot):



IV. Extended Questions

1. In Task 1, the function displayHeader() is declared before main() and defined after main(). What may happen if displayHeader() is called in main() before it is declared?

   If displayHeader() is called before the compiler has seen either its function prototype or its full definition, the program will not compile. The compiler cannot recognize the function name at that point, so it may report an error such as "displayHeader was not declared in this scope." To avoid this problem, write a prototype such as void displayHeader(); before main(), or place the complete function definition before main().

2. Read the following code and answer the questions.

```
#include <iostream>
using namespace std;

int number = 10;   // global variable

void showNumber(int number) {
    cout << "In function: " << number << endl;
}

int main() {
    int number = 5;   // local variable

    showNumber(number);
    cout << "In main (local): " << number << endl;
    cout << "In global: " << ::number << endl;

    return 0;
}
```

Questions:

1) What is the output of this program?

In function: 5
In main (local): 5
In global: 10

2) Why does the program produce this output? Explain the roles of the local variable, the global variable, the function parameter, and ::number.

The global variable number is initialized to 10, but inside main() another local variable with the same name is declared and initialized to 5. The local variable has higher priority within main(), so showNumber(number) passes the local value 5 to the function. In showNumber(int number), the function parameter is also named number, and it receives the argument value 5, so the first output line is 5. The statement cout << number inside main() still refers to the local variable, so the second line is also 5. The expression ::number uses the scope resolution operator to explicitly access the global variable, so the last line prints 10.

V. Summary and Reflections

In this experiment, I practiced the basic use of functions in C++ through eight programming tasks. The tasks covered void functions, data-returning functions, functions with parameters, Boolean functions, multiple function calls, and the use of standard library functions from <cmath>. By writing programs for header output, power calculation, maximum value comparison, circle area calculation, temperature conversion, prime number checking, mathematical function processing, and a menu-driven program, I learned how to divide a program into smaller modules and call the correct function according to the program logic.

The main techniques used in this experiment included function declaration, function definition, parameter passing, return values, conditional statements, loops, switch-case selection, constants, and formatted output with fixed and setprecision(2). The final results show that each function can complete its own task and return or display the expected result. This helped me understand that modular programming can make code clearer, easier to debug, and easier to reuse. During the experiment, I also paid attention to invalid input handling, such as checking menu choices and prime-number input.